

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Machine Learning Project Report

MACHINE LEARNING FOR MECHANICAL SYSTEMS

MASTER OF SCIENCE IN MECHANICAL ENGINEERING – MECHATRONICS AND ROBOTICS

Authors: **Pietro Maria Ghisi** 10801017
Yuxuan Lu 11030379
Erwin Rafael Novicio 10782977

Advisor: Loris Roveda
Academic Year: 2025-26

Contents

Contents ..... **III**

1 Parcel sorting optimization in Matlab **5**

1.1. Introduction 5

1.2. Baseline Environment 5

1.3. RL algorithm 6

1.4. Observation Space 6

1.5. Reward 7

1.5.1. Baseline Reward Function 7

1.5.2. Final Reward Function 8

1.6. Hyperparameters 10

1.7. Results and analysis 11

1.7.1. Simulation n.1: 10 packages, $t_{generation}$ = 0.75 s 12

1.7.2. Simulation n. 2: 500 packages, $t_{generation}$ = 0.75 s 13

1.7.3. Simulation n.3: 10 packages, $t_{generation}$ = 0.50 s 15

2 Cartesian impedance-control robot in Python **17**

2.1. Introduction 17

2.2. 2DOF Mass-Spring-Damper System Dynamics Modelling 18

2.2.1. Schematization 18

2.2.2. Mathematical Modelling: Equation of motion and State-space formulation 18

2.2.3. Mechanical system parameters 19

2.2.4. Chirp signal definition 20

2.3. Semi-brute force method 20

2.3.1. Direct optimization 21

2.3.2. Inverse optimization 22

2.4. Bayesian Optimization 23

2.4.1. Dataset generation 23

2.4.2. Gaussian Process model definition 24

2.4.3. Gaussian Process training 26

2.4.4. Kernel benchmarking and best performance selection 27

2.4.5. Gaussian Process training results and analysis 27

2.4.6. Bayesian Optimization procedure 30
2.4.7. Bayesian Optimization: direct problem 32
2.4.8. Bayesian Optimization: inverse problem 33
2.5. Deep Learning 36
2.5.1. Neural Network architecture 36
2.5.2. Direct NN training 36
2.5.3. Sensitivity analysis of the neural network 38
2.5.4. Results and analysis 39
2.5.5. Maximum dynamic displacement 41
2.5.6. Inverse optimization 42
Bibliography 45

1 Parcel sorting optimization in Matlab

1.1. Introduction

In modern automated warehouse logistics and fulfillment hubs, package singulation is a critical preprocessing phase for automated sorting systems. In this project, bulk packages arriving from unstacking areas or unstructured conveyor feeds must be separated into a continuous single-file stream with consistent longitudinal gaps. Proper singulation is mandatory to allow downstream barcode scanners, weigh-in-motion scales, and high-speed cross-belt sorters to identify and route individual packages accurately without physical jams or scanning collisions.

Traditional singulation relies on fixed-speed multi-belt matrices or rule-based heuristics. However, these methods struggle with the complex friction dynamics, varied parcel dimensions, and irregular physical interactions present in real-world logistic streams. To solve this, this project applies a data-driven Reinforcement Learning (RL) framework using continuous dynamic control with Proximal Policy Optimization (PPO) in MATLAB.

1.2. Baseline Environment

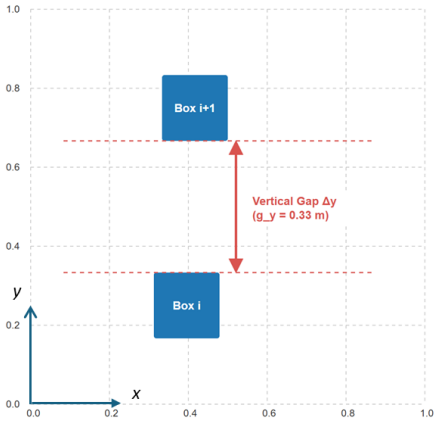


Figure 1

The baseline simulation setup models an Active Motor Surface (AMS) composed of a 5×5 grid of independently controlled motorized conveyor tiles designed for spatial trajectory regulation.

The parcel sorting system specifications include:

- Grid of 5×5 cell array covering a square working area of 0.2 x 0.2 m, with individual cell lateral dimensions of 0.2 m;
- Kinematic actuation consisting of independent translation and rotation of packages per tile;
- Control and action space where each cell directly controls the continuous surface speed $v \in [0.5, 2.2]$ m/s and orientation angle $\theta \in [-\frac{\pi}{4}, \frac{\pi}{4}]$ relative to the main conveying axis, resulting in a continuous action space of dimension 50;
- Stochastic generation of two randomly sized parcels every $t_{generation}$= 0.75 s.

1.3. RL algorithm

Building upon the baseline simulation framework inspired by the work of Roveda et al. (2023), our primary objective was to solve the multi-parcel singulation task on the Active Matrix Surface (AMS) using a deep reinforcement learning architecture. To enable the agent to achieve the expected behaviour, our main contributions focused on designing a normalized observation space scalable to a variable number of packages, formulating a multi-objective composite reward function, and fine-tuning the PPO hyperparameters. The entire framework was implemented in MATLAB using the Reinforcement Learning Toolbox, establishing an end-to-end pipeline that seamlessly integrates policy training, closed-loop batch simulation, and automated statistical post-processing to evaluate physical package separation.

1.4. Observation Space

To construct a functional reinforcement learning environment from the provided skeleton code, a custom observation space had to be conceptualized and implemented from scratch. The principal design challenge was representing an arbitrary number of moving parcels without altering the state dimension during an episode.

By decoupling the physical matrix state from the dynamic parcel generation sequence, this design ensures numerical stability, provides clear spatial localization for the PPO agent, and prevents dimension mismatches across variable-length simulations.

At discrete time step k (where $t = k \cdot \Delta t$, with Δt = 0.01 s) the full observation state $s(k) \in \mathbb{R}^{100}$ received by the RL agent is a 100-dimensional column vector and it is constructed by concatenating the 4-dimensional local feature vectors $c_m(k) \in \mathbb{R}^4$ of each grid cell $m = 1, 2, \ldots, 25$:

$$s(k) = \begin{bmatrix} c_1(k) \\ c_2(k) \\ \vdots \\ c_{25}(k) \end{bmatrix}$$

For any cell m, the local state vector is structured as:

$$c_m(k) = \begin{bmatrix} \hat{x}_m(k) \\ \hat{y}_m(k) \\ \hat{d}_m(k) \\ o_m(k) \end{bmatrix}$$

A binary flag denoting whether one or more active parcels currently reside within the spatial boundaries of cell m:

$$o_m(k) = \begin{cases} 1, & if\ P_m(k) \neq \emptyset \\ 0, & if\ P_m(k) = \emptyset \end{cases}$$

where $P_m(k)$ is the set of parcels present on the cell m at time step k.

If the cell is unoccupied ($o_m(k)$ = 0), all the elements are set to zero:

$$c_m(k) = [0, 0, 0, 0]^\top$$

On the other hand, in the case where the cell is occupied ($o_m(k)$ = 1), the physical features are non-dimensionally scaled relative to the total surface dimension $L_{max} = 5d_{AMS}$ = 1.0 m, where the tile size is d_{AMS} = 0.2 m.

The normalized features are given by:

$$\hat{x}_m(k) = \frac{x_i(k)}{L_{max}};\ \hat{y}_m(k) = \frac{y_i(k)}{L_{max}};\ \hat{d}_m(k) = \frac{d_i}{2d_{AMS}}$$

where $\hat{x}_m(k)$, $\hat{y}_m(k)$, and $\hat{d}_m$(k) are the normalized transverse position, longitudinal position and parcel radius, respectively.

1.5. Reward

1.5.1. Baseline Reward Function

The baseline formulation [1] evaluates parcel singulation downstream across a designated 1.0 m exit region using a non-linear local score system based on arctangent clearance:

- Step 1: Signed gap calculation:

$$d_k = y_{min}(p_{k+1}) - y_{max}(p_k)$$

- Step 2: Arctangent local score mapping:

$$l_{s_k} = \min(10 \cdot K \cdot \arctan(d_k) + K, 6)$$

- Step 3: Worst-case bottleneck evaluation:

$$r_t = \min_{1 \le k \le K-1} l_{s_k}$$

- Step 4: Sparse clearance bonus:

$$r_t \leftarrow r_t + 20 \; if \; d_k > 0 \; \forall k \; and \; K \geq 3$$

However, the baseline reward imposes several limitations. It is proven that the baseline formulation is effective for low-density or static regimes, but it presents several bottlenecks when applied to dense, continuous parcel ingestion:

- Evaluation only occurs in the downstream region after parcels cross y = 1.0 m. While parcels are traveling across the AMS grid, the agent receives zero intermediate feedback.
- The non-linear $\arctan(d_k)$ mapping saturates rapidly for larger separations, yielding vanishing gradients that slow down policy gradient update steps during early training phases.
- Using a strict $\min(\cdot)$ operator evaluates only the single worst-case pair (p_k, p_{k+1}). If multiple distinct parcel collisions or jams occur simultaneously elsewhere on the surface, their penalties are masked out, preventing multi-contact credit assignment.

1.5.2. Final Reward Function

To resolve the limitations given by the baseline reward, the new reward function was redesigned into a fully dense, linearly responsive multi-objective scalar reward evaluated at every simulation step Δt = 0.01 s. The total instantaneous reward is defined as:

$$R(t) = R_{progress}(t) - R_{collision}(t) + R_{exit}(t)$$

To encourage continuous longitudinal transport along the AMS, the Progress Reward $R_{progress}(t)$ is designed to provide a dense positive feedback while avoiding reward exploitation. Designed specifically for high-density package processing, the new reward structure balances forward package throughput, safe inter-parcel separation, and total system output.

In the baseline reward function, there was no feedback while parcels are on the AMS grid, and in order to solve this, a continuous longitudinal displacement incentive is introduced for every active package:

$$R_{progress}(t) = \sum_{i=1}^{N_{active}(t)} \max(0, w_p \cdot \Delta \mathrm{y_i}(t)) = \sum_{i=1}^{N_{active}(t)} \max(0,120 \cdot (\mathrm{y_i}(t) - y_i(t - \Delta t))$$

where $N_{active}(t)$ is the total number of parcels currently present on the system, the term $\max\left(0, w_p \cdot \Delta y_i(t)\right)$ represents the longitudinal forward displacement along the belt accompanied with the w_p = 120, this keeps parcels moving ahead and stops the agent from making them stall, move backward, or spin in circles. The condition $0 < y_i(t) \leq 1.0\, m$ is posed in order to impose that the active reward domain is bounded strictly to the AMS grid, ensuring that parcels driven passively by the treadmill do not generate spurious rewards.

To prevent physical overlaps, jamming, and damage during dense package ingestion, a continuous Collision Penalty term is calculated to penalize parcels whenever their center distance falls below the safety clearance.

The baseline reward function has the term $r_t = \min_k l_{s_k}$ which ignores secondary collisions happening concurrently. By introducing the collision penalization term, penalties now are accumulated across all unique active pairs on the matrix:

$$R_{collision}(t) = \sum_{i=1}^{N_{active}(t)} \sum_{j=i+1}^{N_{active}(t)} \Phi(i,j)$$

This ensures that the credit assignment accounts for the global collision footprint of the high-density streams rather than a single isolated pair.

On the other hand, the $\arctan(d_k)$ from the baseline reward imposes a hard asymptotic bound on the penalty magnitude during deep overlaps, for this a pairwise collision penalty term $\Phi(i,j)$ was introduced given by:

$$\begin{cases} w_c \cdot p_y(i,j) + \lambda_c, & if\ \left|y_i(t) - y_j(t)\right| < d_{safe}, ij\ and\ y_i, y_j > 0 \\ 0, & otherwise \end{cases}$$

This is activated whenever the longitudinal center-to-center distance falls below the minimum safe physical clearance threshold d_{safe}, ij, giving therefore a linear overlap depth penalty scaled by w_c, plus a step penalty λ_c. The linear depth scaling produces constant gradient magnitudes during physical overlap, aggressively driving colliding parcels apart, restoring a spatial clearance between adjacent packages.

The terms in the equation are:

- $d_{safe,ij} = \frac{d_i + d_j}{2}$ is the sum of radii of parcels i and j;
- $p_y(i,j) = d_{safe}, ij - \left|y_i(t) - y_j(t)\right|$ is the longitudinal overlap;
- w_c = 250 is the penalty scaling relative to the severity of the physical overlap;
- λ_c = 8.0 acts as a discrete step-wise baseline penalty applied as soon as proximity safety limits are violated.

To incentivize the agent to complete the singulation task by driving parcels off the active surface onto the downstream exit conveyor, the Exit Reward was designed to reward the agent whenever each parcel successfully crosses the sorting exit line.

From the baseline reward, the sparse +20 bonus requires global perfection, when $d_k \geq 0 \; \forall k$, simultaneously across all parcels. The bonus is replaced with a cumulative throughput bonus tied directly to exit completion:

$$R_{exit}(t) = w_e \cdot N_{exited}(t)$$

where $N_{exited}(t)$ is the cumulative count of packages that have successfully crossed the exit boundary up to time step t, and $w_e = 350$ is the exit reward weight multiplier.

Overall, the $R_{exit}(t)$ grants incremental credit whenever any individual package completes singulation and exits onto the downstream conveyor, maintaining strong gradient incentive throughout dense multi-parcel episodes. This encourages the agent to maximize overall sorting throughput and speed.

1.6. Hyperparameters

Compared to the baseline configuration, the tuned agent hyperparameters emphasize faster convergence and long-horizon credit assignment. The actor and critic learning rates were increased, with respect to their original values, to accelerate policy updates and value function tracking. The experience horizon was expanded from 500 to 2048 steps to cover the full physical parcel flow across the AMS matrix. Furthermore, the discount factor was raised to $\gamma = 0.998$ to extend the agent's effective planning horizon, while the clip factor (0.04) and entropy weight (0.02) were tuned to strike an optimal balance between monotonic policy improvement and robust exploration.

The hyperparameters are shown below:

Hyperparameter	**Value**
Learning rate actor	5e-4

Learning rate critic	9e-3
Experience Buffer	2048
Clip factor	0.04
Entropy Weight	0.02
GAE factor	0.95
Discount factor	0.998

1.7. Results and analysis

The training of the RL agent resulted in the following plot:

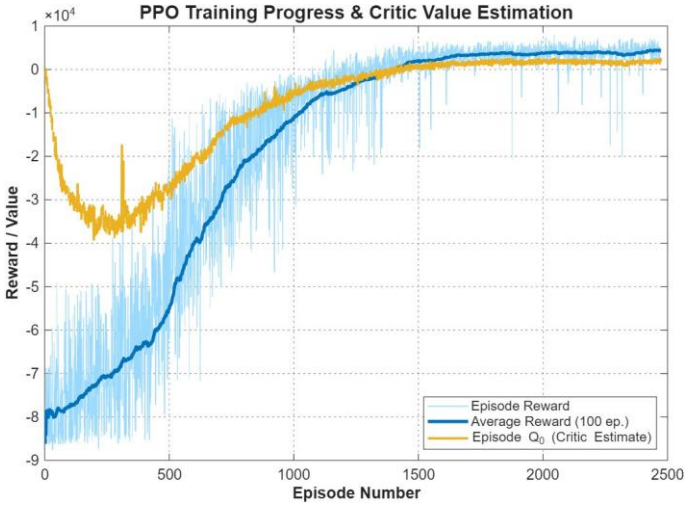


Figure 2

The training curves demonstrate a robust and monotonic convergence profile across the 2500 episodes. During the initial exploration phase (Episodes 0–500), random actuation caused frequent parcel collisions, incurring severe penetration penalties that yielded low episode rewards $\approx -8 \times 10^4$. Between Episodes 500 and 1500, the PPO agent underwent rapid policy improvement, steadily reducing collision rates and mastering coordinated velocity and angular adjustments to singulate parcel pairs. Beyond Episode 1500, the 100-episode average reward stabilized in the positive domain $\approx +4 \times 10^3$ with minimal variance. Notably, the Critic's initial state value estimation (Episode Q_0) tightly tracks the actual moving average reward, confirming accurate value function approximation without value divergence or overestimation bias. Overall the training process worked out well.

1.7.1. Simulation n.1: 10 packages, $t_{generation}$ = 0.75 s

To evaluate the operational robustness of the trained PPO agent under high-density parcel streams, a Monte Carlo simulation benchmark consisting of 100 independent episodes was conducted. In each simulation, 2 packages were continuously ingested onto the 5x5 Active Matrix Surface (AMS) at a generation interval of $t_{generation}$ = 0.75 s until the total number of 10 packages generated was reached.

The performance across the trial batch was evaluated using three key spatial metrics: global exit success rate, computed as ratio of successful pair with gap over the number of total pairs generated, gap distributions among the simulations, and a representative single-episode gap breakdown.

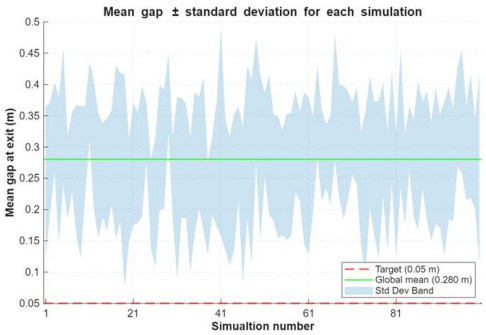


Figure 3

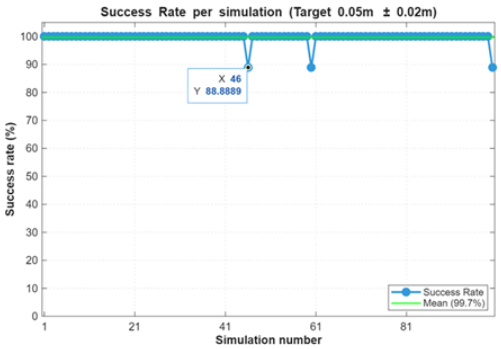


Figure 4

Across all 100 simulations, the overall global mean exit gap was 0.280 m comfortably exceeding the minimum arbitrary set target threshold (0.05 m). The lower bound of the standard deviation band remains consistently above d = 0.08 m across virtually all runs. This confirms that the linear collision penalty successfully trains the policy to prioritize safe clearance over risky, overly tight packing. The standard deviation band spans between roughly 0.12 m and 0.45 m. This variation shows that the agent does not enforce a rigid, static speed profile; instead, it adaptively modulates tile speeds based on individual parcel dimensions and entry positions.

In the Figure 4, the agent achieved a global mean success rate of 99.7% across the 100 simulation runs. Across the entire 100-run dataset, only three isolated simulations (Simulations #46, #59, and #99) dipped below 100% success, recording a success rate of 88.89%. This corresponds to exactly one package pair out of nine failing to meet the tight target threshold due to unfavorable stochastic initialization (such as maximum parcel size combined with near-zero initial lateral offset). In all other 97 episodes, 100% of package pairs exited successfully.

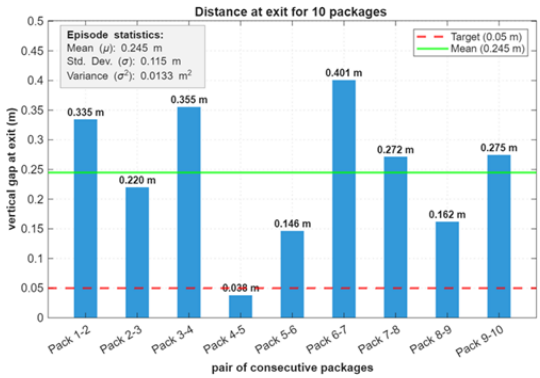


Figure 5

The spatial separation dynamics is represented in the Figure 5, a random single 10-package episode was extracted for detailed inspection. The maximum gap occurred between packages 6–7 (0.401 m), where the agent decelerated the trailing package to resolve upstream crowding while the minimum gap occurred between packages 4–5 (0.038 m). While slightly below the 0.05 m nominal target, a gap of $d > 0$ strictly guarantees that no physical contact or overlapping occurred during exit**.**

1.7.2. Simulation n. 2: 500 packages, $t_{generation} = 0.75$ s

To evaluate the policy under prolonged continuous throughput and steady-state conditions, the simulation volume was scaled up to a total stream of **500 packages** generated continuously at $t_{generation} = 0.75$ s.

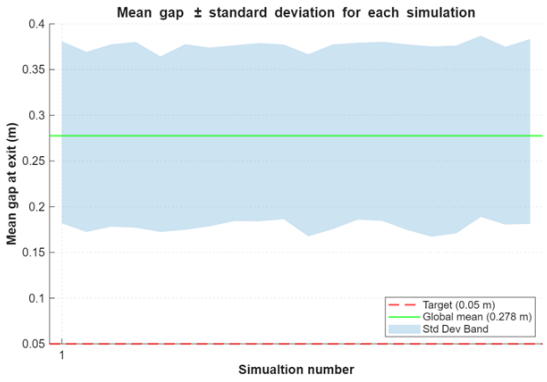


Figure 6

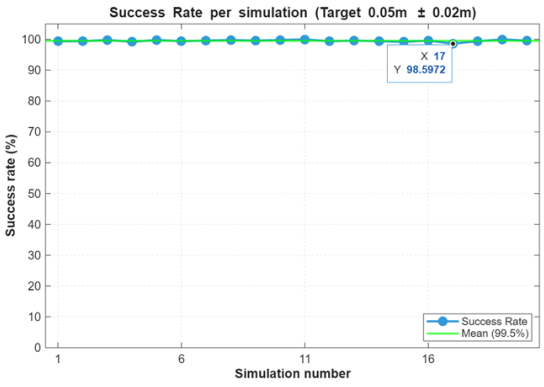


Figure 7

Comparing the multi-simulation gap plot across 500 packages against shorter 10-package trial batches, it can be observed in the plot in the Figure 6 that the upper and lower boundaries of the standard deviation band transition into a smooth, near-

constant horizontal envelope. Scaling to 500 packages averages out the local transient noise of individual 10-package episodes. This demonstrates that the agent enforces a stationary stochastic state, preventing error accumulation over extended operating durations. The overall global mean exit gap across the 500-package evaluation settles at 0.278 m, which is nearly identical to the smaller test batches, demonstrating a scale-invariant performance. The entire standard deviation envelope remains well above the target threshold, confirming that physical package clearance is strictly preserved throughout long-duration streams.

In the Figure 7 the policy maintains a high global mean success rate of 99.5% across all test runs. Unlike smaller test batches where an isolated misaligned pair could drop an individual 10-package run's success rate to 88.9%, the large package count amortizes localized timing variations. Across the entire simulation set, even the minimum observed success rate remains exceptionally high at 98.5972% (Simulation #17).

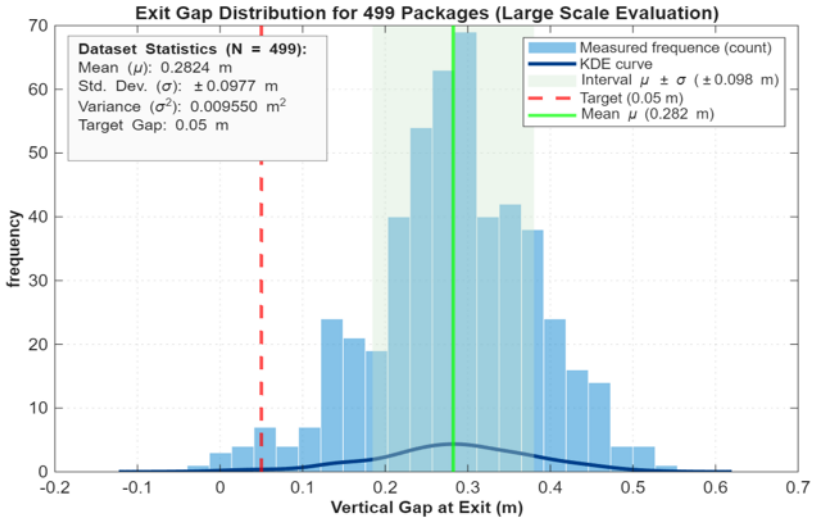


Figure 8

The Figure 8 shows the complete frequency histogram and Kernel Density Estimation (KDE) for the N = 499 consecutive exit gaps recorded during a 500-package stream. The overall average separation distance at exit is 0.2824 m, which is over five times larger than the minimum target clearance of 0.05 m. This indicates that the policy successfully prioritizes maintaining a safe spatial buffer to prevent downstream package collisions. The majority of package exits are tightly grouped within the 1-sigma confidence band, showing predictable spatial control under stochastic generation conditions. It can also be seen from the plot that the KDE curve overlaid on the histogram illustrates a smooth, unimodal distribution centered at 0.282 m. Finally, the left tail of the distribution rapidly drops off before reaching zero. The absence of negative gap values confirms that no physical overlaps or

package jams occurred during matrix egress across all 499 consecutive package pairs.

1.7.3. Simulation n.3: 10 packages, $t_{generation} = 0.50$ s

To determine the operational boundary and breaking point of the policy, the package ingestion interval was shortened from 0.75 s to 0.50 s (33.3% increase in temporal throughput).

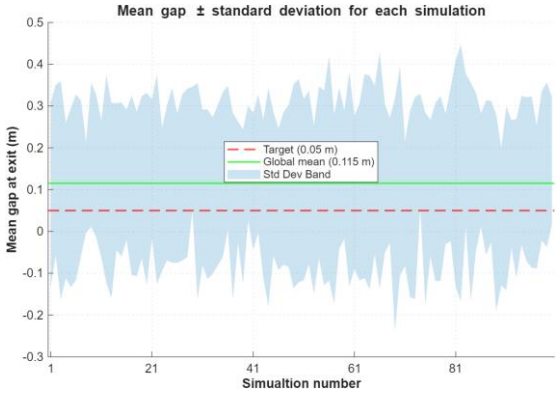


Figure 9

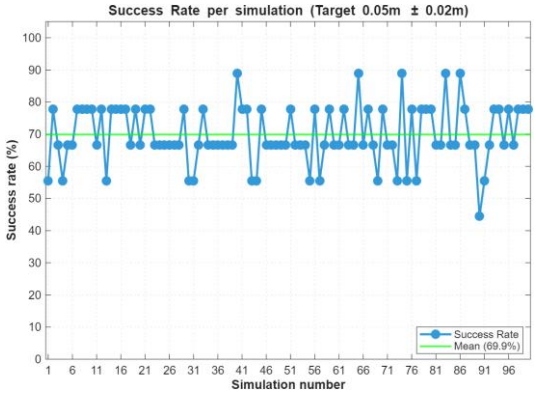


Figure 10

In the Figure 9, the global average exit gap decreases sharply to 0.115 m, compared to 0.280 m at 0.75 s. The light blue standard deviation band extends significantly below the zero-gap threshold, reaching values down to -0.20 m, which would mean a physical collision (negative gap). This demonstrates that at 0.50 s, incoming packages arrive faster than the physical speed limits of the 5x5 matrix actuators can clear them, causing an upstream queuing bottleneck.

In can be observed in the Figure 10 that the global mean success rate falls from 99.7% down to 69.9%. Individual simulation success rates fluctuate between a minimum of 45% and a peak of 88%, approximately. No single episode achieves 100% success, confirming that the 0.50 s regime consistently exceeds the surface's physical handling capacity.

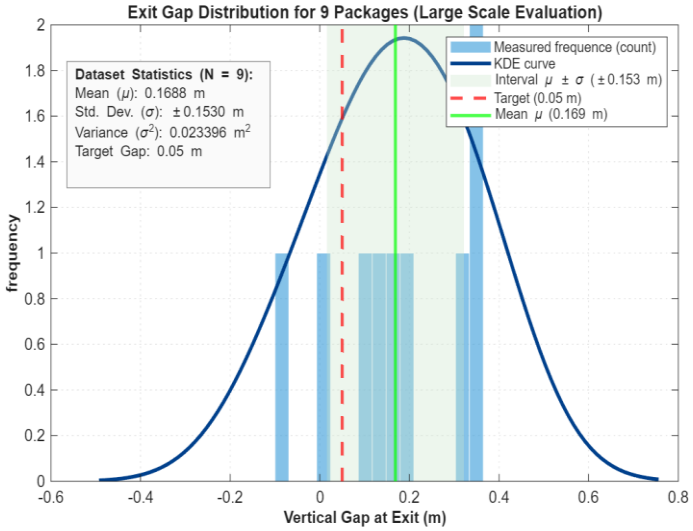


Figure 11

In the Figure 11 the plot shows the statistical distribution for a single 10-package episode (N = 9 consecutive exit gaps). This wide dispersion of the standard deviation indicates that the grid overcrowding forces the agent into highly erratic, un-smooth speed adjustments. In contrast to the 0.75 s baseline, the histogram features an exit gap bar in the negative region around -0.10 m. A negative distance confirms that under 0.50 s ingestion, package arrival rates exceed the actuation speed limits of the 5x5 matrix cells, resulting in unavoidable physical collisions.

In summary, reducing the package generation interval to 0.50 s leads to a noticeable performance degradation across the 100 simulation runs. Although the policy manages to maintain a positive global mean exit gap, the increased spatial variance and occurrence of negative gaps indicate that the agent struggles under this higher ingestion frequency. Retraining the PPO agent explicitly under these high-density, 0.50 s operational conditions, or expanding the state observation window, could allow the policy to adapt its control strategy and mitigate upstream bottlenecks.

2 Cartesian impedance-control robot in Python

2.1. Introduction

This project addresses the application of several Machine Learning techniques for the analysis and optimization of a nonlinear and underactuated mechatronic system. The system under study is a Cartesian impedance-controlled robot mounted on a compliant base, modelled as a 2DOF mass-spring-damper system connected in series.

The system exhibits the following Degrees of Freedom:

- Robot End-Effector (Actuated DOF): Represented by the mass m_r, whose position can be actively controlled via a control law within a feasible physical boundary.
- Robotic Base (Passive DOF): Represented by the base mass m_b, which is completely underactuated and subject to purely passive dynamics, compliance and damping of the structural foundation.

The main objective is to achieve the dynamic extension of the robot's workspace.

2.2. 2DOF Mass-Spring-Damper System Dynamics Modelling

2.2.1. Schematization

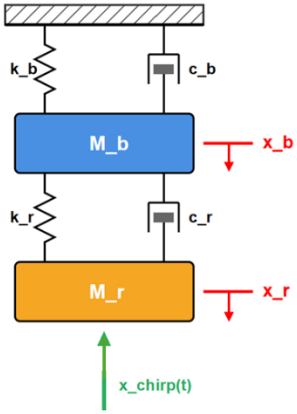


Figure 12

2.2.2. Mathematical Modelling: Equation of motion and State-space formulation

To solve the dynamics [2] of the given problem, the Lagrangian approach was used, thus the following equation was written:

$$\frac{d}{dt}\left(\frac{\partial T}{\partial \dot{\bar{x}}}\right) - \frac{\partial T}{\partial \bar{x}} + \frac{\partial D}{\partial \dot{\bar{x}}} + \frac{\partial V}{\partial \bar{x}} = \bar{0}$$

Kinetic energy:

$$T = \frac{1}{2} m_b \dot{x_b}^2 + \frac{1}{2} m_r (\dot{x_b} + \dot{x_r})^2$$

Damping energy:

$$D = \frac{1}{2} c_b \dot{x_b}^2 + \frac{1}{2} c_r \dot{x_r}^2$$

Potential energy:

$$V = \frac{1}{2} k_b x_b^2 + \frac{1}{2} (x_r - x_{cmd})^2$$

Finally, the equation of motion is given by:

$$\begin{cases} (m_b + m_r)\ddot{x_b} + m_r x_r + c_b \dot{x_b} + k_b x_b = 0 \\ m_r \ddot{x_r} + m_b \ddot{x_b} + c_r \dot{x_r} + k_r x_r - k_r x_r = 0 \end{cases}$$

We would like to write the equation of the motion in the following form:

$$\dot{\bar{x}} = A\bar{x} + Bu$$

The state vector is defined as:

$$\bar{x} = \begin{bmatrix} \dot{x_b} \\ \dot{x_r} \\ x_b \\ x_r \end{bmatrix}$$

The input is defined as:

$$u = x_{cmd}$$

The system matrices are:

$$A = \begin{bmatrix} -\frac{c_b}{m_b} & \frac{c_r}{m_b} & -\frac{k_b}{m_b} & \frac{k_r}{m_b} \\ \frac{c_b}{m_b} & -\left(\frac{c_r}{m_b} + \frac{c_r}{m_r}\right) & \frac{k_b}{m_b} & -\left(\frac{k_r}{m_b} + \frac{k_r}{m_r}\right) \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \qquad B = \begin{bmatrix} -\frac{k_r}{m_b} \\ \frac{k_r}{m_b} + \frac{k_r}{m_r} \\ 0 \\ 0 \end{bmatrix}$$

2.2.3. Mechanical system parameters

The viscous damping can be calculated as:

$$c = 2h\omega$$

where:

$$\omega = \sqrt{\frac{k}{m}}$$

is the natural frequency of the mechanical system.

Assuming that in the 2DOF system, the resonant peaks are well-separated, the viscous damping coefficients for the base and robot actuator can be written as:

$$c_b = 2h_b\omega_b$$

$$c_r = 2h_r\omega_r$$

2.2.4. Chirp signal definition

The reference trajectory signal commanding the robot's end effector is modelled as a frequency-modulated sinusoidal sweep, also known as the linear chirp signal, in which the instantaneous frequency varies continuously over the entire time simulation from an initial frequency f_0 to a final frequency f_1.

The linear rate of frequency change is defined as:

$$k = \frac{f_1 - f_0}{T_{sim}}$$

The instantaneous frequency is expressed as:

$$f(t) = f_0 + kt = f_0 + \frac{f_1 - f_0}{T_{sim}}$$

$$x_{chirp}(t) = x_{r,start} + A_{chirp}\cos\big(\phi(t)\big) =$$

The phase angle of the signal can be obtained by integrating the instantaneous angular frequency over time:

$$\phi(t) = \int_0^t \omega(t)\,d\tau = \int_0^t 2\pi f(t)\,d\tau = 2\pi\int_0^t f_0 + k\tau\,d\tau = 2\pi\left(f_0 t + \frac{1}{2}kt^2\right)$$

Finally, the chirp signal can be expressed as:

$$x_{chirp}(t) = x_{r,start} + A_{chirp}\cos\big(\phi(t)\big) = x_{r,start} + A_{chirp}\cos\left(2\pi\left(f_0 t + \frac{1}{2}kt^2\right)\right)$$

where $x_{r,start}$ is the initial displacement bias and A_{chirp} is the amplitude of the oscillation of the robot's end effector.

2.3. Semi-brute force method

Testing every possible combination of robot designs and motor frequencies using a traditional grid search is practically impossible as it would require millions of complex dynamic simulations. On the other hand, simple local optimization methods often get stuck in the wrong solutions because the system's physics are non-linear and complex.

The Semi-Brute Force method is a hybrid computational framework designed to explore non-convex, high-dimensional parameter spaces where analytical closed-form solutions are unavailable. It uses smart sampling combined with a targeted search to explore the system efficiently.

The implementation of the Semi-Brute Force method could be summarized in the following: first, by using the LHS sampling, the parameters are sampled uniformly across the input space dimension, and then by running Differential Evolution (DE) algorithm directly on the numerical differential equation solver, the method discovers the absolute maximum distance the robot end-effector can reach. Because it calculates actual system dynamics step-by-step, it provides an unbiased ground truth to verify target reachability.

2.3.1. Direct optimization

To establish the absolute performance baseline for dynamic reach extension, a forward global optimization was executed via Differential Evolution directly coupled with the high-fidelity RK45 ODE solver. By minimizing the negative peak end-effector displacement, the solver navigated the non-convex 9-dimensional parameter space to discover the system's global reach ceiling without relying on model approximations. The optimization identified a maximum dynamic displacement of 1.5772 m, effectively more than doubling the robot's 1 dof nominal reach. The optimal parameter set revealed clear mechatronic trade-offs: the algorithm minimized damping and base mass while saturating control boundaries and tuning joint stiffnesses to align the system's natural frequencies with the low-frequency chirp excitation. Beyond validating the efficacy of compliant base resonance, this global peak serves as the upper reach limit and ground-truth benchmark for validating downstream machine learning surrogates and inverse target tracking algorithms.

The following values were obtained:

Parameter	Optimal Value
K_b [N/m]	4889.12
K_r [N/m]	2617.95
M_b [kg]	10.0
h_b [-]	0.1
h_r [-]	0.1
f_0 [Hz]	0.0001
f_1 [Hz]	3.0
$A_{\sin}$ [m]	0.2
$x_{r,\text{start}}$ [m]	0.5
Maximum displacement of the robot [m]	1.577218

Figure 13

2.3.2. Inverse optimization

To evaluate target reachability under fixed hardware constraints, an inverse optimization pipeline was implemented using Differential Evolution coupled directly with the high-fidelity RK45 ODE solver. By locking the compliant base hardware properties and defining a target displacement, the solver optimized the remaining six operational decision variables. The optimization achieved an exceptionally accurate peak reach of 1.12 m, yielding a residual square error of 2.68e-10 and an absolute error of just 0.00002 m. To hit this sub-maximal target rather than the global reach ceiling, the algorithm intelligently backed off on excitation energy, reducing the input amplitude and broadening the chirp sweep frequency to pass through resonance more rapidly. This confirms that modulating the active joint's sweep profile allows precise workspace position control on a compliant base, providing a ground-truth baseline for training fast inverse Machine Learning surrogates.

The resulting parameters from the inverse optimization are:

Parameter / Metric	Value
K_b [N/m] *(FIXED)*	4889.12
K_r [N/m] *(OPTIMIZED)*	4315.18
M_b [kg] *(FIXED)*	10.00
h_b [-] *(FIXED)*	0.10
h_r [-] *(OPTIMIZED)*	0.12
f_0 [Hz] *(OPTIMIZED)*	0.0029
f_1 [Hz] *(OPTIMIZED)*	9.35
$A_{\sin}$ [m] *(OPTIMIZED)*	0.17
$x_{r,\text{start}}$ [m] *(OPTIMIZED)*	0.49
Target displacement [m]	1.12
Achieved peak displacement [m]	1.12002

Figure 14

2.4. Bayesian Optimization

2.4.1. Dataset generation

The parameters for the Bayesian Optimization are defined in the following:

$$\theta = [k_b\ k_r\ m_b\ h_b\ h_r\ f_0\ f_1\ A_{chirp}\ x_{r,start}]$$

The method to generate the samples is the Latin Hypercube Sampling, which guarantees uniform samples in each parameter range:

$$X = \begin{bmatrix} k_b^1 & k_r^1 & m_b^1 & h_b^1 & h_r^1 & f_0^1 & f_1^1 & A_{chirp}^1 & x_{r,start}^1 \\ . & . & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & . & . \\ k_b^{n_s} & k_r^{n_s} & m_b^{n_s} & h_b^{n_s} & h_r^{n_s} & f_0^{n_s} & f_1^{n_s} & A_{chirp}^{n_s} & x_{r,start}^{n_s} \end{bmatrix}$$

.

Successively the samples generated from the LHS were inserted to the ODE solver, therefore the peaks maximum peak displacements could be obtained:

$$Y = \begin{bmatrix} y_{peak}^{1} \\ . \\ . \\ . \\ y_{peak}^{n_s} \end{bmatrix}$$

In order for the Gaussian Process model to be able to predict unseen data, the whole parameter set will be divided into train and test samples:

$$(X, Y) \rightarrow (X_{train} \in \mathbb{R}^{n_{s,train} \times n_p}, Y_{train} \in \mathbb{R}^{n_{s,train}}) \;\&\; (X_{test} \in \mathbb{R}^{n_{s,test} \times n_p}, Y_{test} \in \mathbb{R}^{n_{s,test}})$$

2.4.2. Gaussian Process model definition

Input features often have vastly different units and physical scaled, for this reason a proper normalization is applied, to ensure equal sensitivity across all physical parameter dimensions during the Gaussian Process Regression

A StandardScaler is initialized and fitted exclusively on the training set to compute feature-wise empirical means and standard deviations:

$$X_{train,scaled} = \frac{X_{train} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

where:

$$\boldsymbol{\mu} = [\mu_1, \ldots\ldots, \mu_{n_s}]$$

$$\boldsymbol{\sigma} = [\sigma_1, \ldots\ldots, \sigma_{n_s}]$$

To maintain rigorous evaluation boundaries and prevent data leakage, the test dataset X_{test} and the physical search bounds $[l_b, u_b]$ are transformed using the training set statistics $(\boldsymbol{\mu}, \boldsymbol{\sigma})$. This maps the physical parameter domain into a standardized feature space optimized for GP length-scale convengence.

The normalization for each parameter is computed as:

$$x_{j,scaled} = \frac{x_j - \mu_j}{\sigma_j}$$

The raw physical range of each input parameters, for each dimension j is defined as:

$$\Delta x_j = u_j - l_j$$

The input span must also be scaled with respect to the standard deviation, therefore:

$$\Delta x_{j,scaled} = \frac{\Delta x_j}{\sigma_j} = \frac{u_j - l_j}{\sigma_j}$$

In the Gaussian Process modelling, in order to determine which parameter affects/weighs the most, length scales are defined.

The length-scale hyperparameter of the Gaussian Process determines the characteristic distance over which the correlation between two input points decreases. Consequently, it provides an indication of the smoothness of the response surface: a small length-scale allows the GP to model rapidly varying responses, whereas a larger length-scale results in a smoother surrogate model.

To investigate the effect of the assumed response smoothness, three initial length-scale configurations were considered. The heuristic length scales were defined as fractions of the span of each input variable:

$$l_j^{short} = 0.15\Delta x_{j,scaled}$$

$$l_j^{medium} = 0.5\Delta x_{j,scaled}$$

$$l_j^{long} = 1.0\Delta x_{j,scaled}$$

The three configurations represent different prior assumptions about the smoothness of the unknown response surface. The short length-scale configuration allows for more rapid variations, the medium configuration assumes moderate smoothness, and the long length-scale configuration assumes that the response varies more gradually throughout the design space.

Separate length scales were assigned to the individual input dimensions, resulting in an Automatic Relevance Determination (ARD) formulations. Thus, the kernel can assign a different characteristic correlation distance to each physical parameter. During GP training, these initial length scales are subsequently optimized by maximizing the log marginal likelihood, subject to the specified hyperparameter bounds. Therefore, the values defined above should be interpreted as initial hyperparameter values rather than the final length scales of the trained GP.

To identify a suitable covariance model for the Gaussian Process surrogate, several kernel configurations where benchmarked. The objective of this analysis was to investigate how different assumptions regarding the smoothness, correlation length, signal amplitude, and observation noise affect the predictive performance of the Gaussian Process.

Kernel general structure:

$$k(x, x') = \sigma^2 k_{basis}(x, x') + \sigma_n^2 \delta(x, x')$$

where k_{basis} denotes the selected base kernel/basis function (RBF, Matern, ec…), σ^2 is the signal variance, and σ_n^2 represents the observation noise variance (considered as white noise).

The signal and noise variances were initialized using the standard deviations of the training outputs.

Several kernel configurations were defined:

- Matern
- RBF
- Exponential

The different kernel configurations were selected to investigate progressively different assumptions about the regularity of the underlying response surface. In particular the Matern kernel provides smoother response, while the exponential covariance function corresponds to the limiting case of the Matern and represents less smooth variations. The RBF kernel represents an infinitely differentiable, highly smooth surface.

2.4.3. Gaussian Process training

The different covariance functions introduced previously were benchmarked using the *GaussianProcessRegressor* implementation from Scikit-learn. For each candidate kernel, an independent Gaussian Process Model was initialized and trained using the training dataset.

For each kernel configuration k_i, the Gaussian Process was defined as

$$f(\theta) \sim GP\big(m(\theta), k_i(\theta, \theta'; \delta)\big)$$

where δ denotes the trainable hyperparameters:

$$\delta = [l, \sigma^2, \sigma_n^2]$$

The models were trained by maximizing the logarithm of the marginal likelihood with respect to these hyperparameters:

$$\delta^* = \arg\max_{\delta} \log p(Y|X, \delta)$$

In particular, the signal variance, length scales, and noise variance were allowed to vary from their initial values during optimization.

To reduce the dependence of the optimization procedure on the initial hyperparameter values, the optimizer was restarted ten times from different initial

conditions. The solution corresponding to the best optimized marginal likelihood was retained as the final set of hyperparameters for each kernel configuration.

2.4.4. Kernel benchmarking and best performance selection

The different kernel configurations were subsequently compared using predictive metrics on both the training and test datasets. The configuration providing the most appropriate predictive performance was then selected as the basis for the subsequent Bayesian Optimization stage.

The performance of each Kernel is evaluated on the train and test set, using the following KPIs:

$$RMSE = \sqrt{\frac{1}{N}\sum(y - \hat{y})^2}$$

$$MAE = \frac{1}{N}\sum|y - \hat{y}|$$

$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \overline{y})}$$

After training, the optimized kernel was extracted from each Gaussian Process model. The resulting metrics and optimized hyperparameters were stored in a benchmark table, allowing the predictive accuracy and the resulting covariance structure of each candidate model to be compared systematically. The kernel providing the most favorable combination of low prediction error and high R^2 was subsequently selected as the surrogate model for the Bayesian Optimization stage.

2.4.5. Gaussian Process training results and analysis

The predictive capabilities of the optimized Gaussian Process surrogate were evaluated on an independent test dataset using a Parity Plot. Among the candidate kernel architectures and lengthscale heuristics, the Matérn 3/2 kernel with short lengthscale initialization (Matern_32_short) achieved superior performance, yielding a coefficient of determination of $R^2 = 0.958$ on unseen test evaluations.



Figure 15

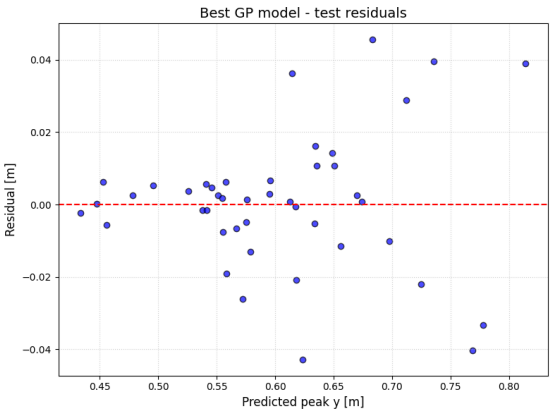


Figure 16

To complement the parity analysis, a residual plot was generated for the test set to evaluate regression bias and variance characteristics across operating amplitudes. Overall the residual distribution confirms that the surrogate model is well-calibrated across all regimes, with residual limits ($\leq 0.04\ m$) remaining well within tolerable limits for guiding Bayesian Optimization.

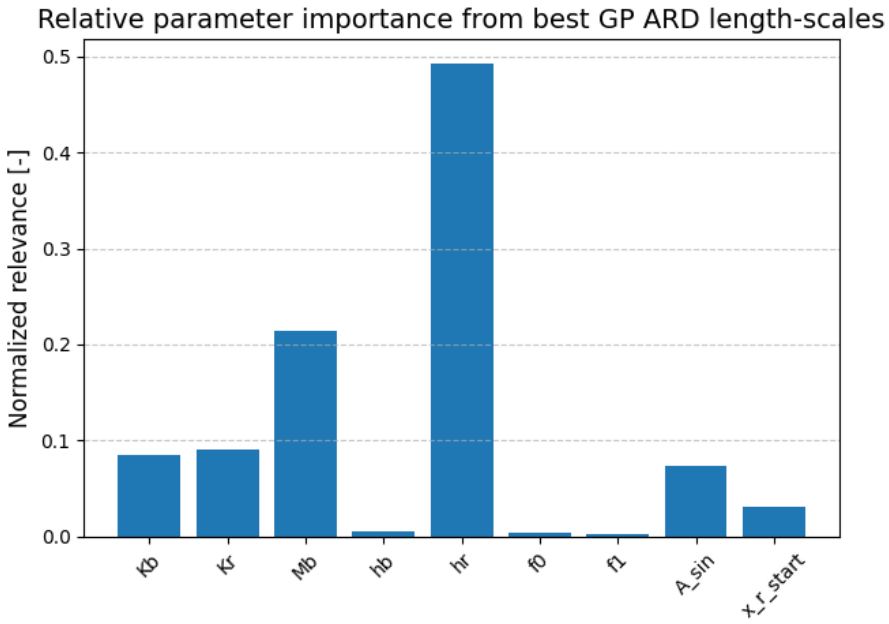


Figure 17

To quantify the relative influence of each parameter on the maximum displacement response, feature importance metrics were derived from the

optimized Automatic Relevance Determination (ARD) length-scales of the best-performing Gaussian Process surrogate.

Because the length-scale l_i is inversely proportional to the spatial sensitivity of parameter θ_i, normalized relevance weights were formulated as the following:

$$\bar{w}_i = \frac{1}{w_i \sum_j \frac{1}{w_j}}$$

As shown in the parameter importance breakdown, the system response is predominantly governed by the active robot joint damping h_r, which accounts for approximately 49%of the total predictive relevance. The secondary driver is base mass M_b (21%), which directly controls the natural frequency alignment between the dynamic base and the arm. Structural stiffness terms K_b and K_r, drive amplitude A_{sin} are of moderate importance (8% each), whereas base damping h_b and frequency sweep bounds show minimal sensitivity (less than 1%). These sensitivity insights confirm that controlling dynamic energy dissipation and base mass coupling is critical for maximizing reach extension in compliant robotic platforms.

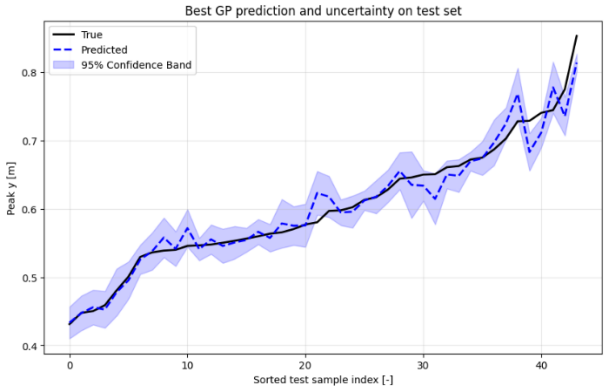


Figure 18

To evaluate the predictive accuracy and uncertainty calibration of the optimized Gaussian Process Regression (GPR) surrogate model, predictions were generated on an unseen test set. The model's predictive uncertainty was quantified via a 95% confidence interval. As demonstrated in the plot, the predicted mean exhibits strong agreement with the true ODE simulation outputs across the operational spectrum. Furthermore, the true response remains consistently bounded within the blue shaded uncertainty envelope, confirming that the GP model is well-calibrated. A slight increase in variance is observed at higher peak displacements, reflecting the increased sensitivity of the dynamic system as it approaches resonant energy bounds. Overall, the surrogate successfully replaces computationally expensive ODE integrations with fast, probabilistic predictions suitable for real-time optimization.

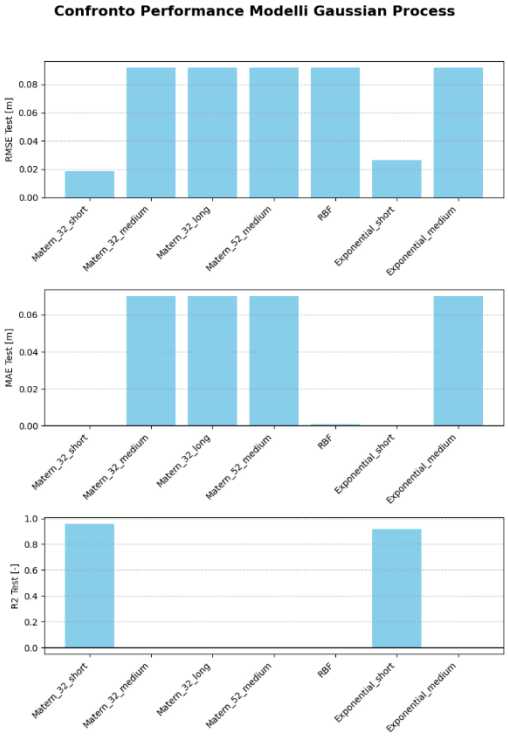


Figure 19

To select the optimal covariance structure for the surrogate model, a comparative benchmark was conducted across seven Gaussian Process kernel variants (Matérn 3/2, Matérn 5/2, RBF, and Exponential) combined with short, medium, and long initial length-scale heuristics. Performance on the unseen test set was evaluated using RMSE, MAE, and the R^2. As illustrated in the multi-panel comparison, the **Matern_32_short** kernel outperforms all alternative configurations, achieving an exceptional fit with RMSE = 0.018 m, MAE = 0.012 m, and R^2 = 0.96. In contrast, configurations initialized with medium or long length-scales severely underfit the training data. This failure occurs because large initial length-scales over-smooth the kernel space during likelihood maximization, preventing the model from resolving localized dynamic resonance. Consequently, the Matérn 3/2 kernel with short initial hyperparameter bounds was selected as the core surrogate architecture for fast target reach predictions.

2.4.6. Bayesian Optimization procedure

Before performing Bayesian Optimization, the design variables were transformed to a normalized search space using a Min-Max scaling procedure (MinMaxScaler). This transformation was applied because the optimization parameters have different physical ranges and units, which can lead to an undesirable imbalance in the numerical representation of the search space.

The Bayesian Optimization search space was subsequently defined directly in the normalized domain $[0,1]^d$, where d is the number of the optimization parameters.

Thus, the Bayesian Optimization procedure is performed in the normalized domain, while the final optimal solution is interpreted in terms of the original physical design parameters using the inverse Min-Max transformation.

The objective function guides the Bayesian Optimization algorithm to obtain the optimal parameters and it is defined based on the task to be performed.

For the optimization, the function was defined as the dynamic maximum peak displacement that can be reached by the robot:

$$\max_{\theta} f(\theta)$$

$$f(\theta) = \max_{t\in[0,T]} y(t;\theta) = \max_{t\in[0,T]} x_b(t;\theta) + x_r(t;\theta)$$

After benchmarking the different Gaussian Process kernel configurations, the best-performing Gaussian Process model was selected as the basis for the subsequent Bayesian Optimization stage. The optimized hyperparameters obtained during the kernel benchmarking phase were extracted from the selected model and used as initial values for the Gaussian Process surrogate employed during Bayesian Optimization.

The resulting values were then used as the starting hyperparameters for the Gaussian Process surrogate in the Bayesian Optimization procedure. In this way, the surrogate model does not start from arbitrary hyperparameter values, but from a covariance model that has already been optimized using the available training data.

This initialization provides a data-informed starting point for the subsequent optimization and preserves the kernel characteristics identified during the preliminary benchmarking stage. The hyperparameters remain trainable during Gaussian Process optimization and can therefore be further adjusted if required by the Bayesian Optimization procedure.

In the Bayesian Optimization, we would like to solve the following:

$$\max_{\theta\in\chi} f(\theta)$$

The parameters will be optimized after a certain number of defined steps.

The Bayesian Optimization of the system parameters is performed as the following steps:

1. **Initial Random Sampling**
   By selecting m points, initial dataset will be created:
   $$D_m = \{(\theta_i, y_i)\}_{i=1}^{m}$$
2. **Building the Prior GP** (with the previously trained GP model)
   The previously trained GP will be used as the prior model
   $$f(\theta) \sim GP(m(\theta), k(\theta, \theta'))$$
   where the $m(x)$ is the mean function and it is supposed to be 0.
3. **BO LOOP**
   3.1) GP Posterior
   $$f(\theta) \mid D_t \sim GP(\mu_t(\theta), \sigma_t^2(\theta))$$
   3.2) Acquisition Function
   $$EI(\theta) = \mathbb{E}[\max(0, f^* - f(\theta))]$$
   3.3) Next point
   $$\theta_{t+1} = \arg\max_{\theta \in \chi} EI(\theta)$$
   3.4) Evaluate the real dynamic system
   $$y_{t+1} = f(\theta_{t+1})$$
   3.5) Update the dataset
   $$\mathbb{D}_{t+1} = \mathbb{D}_t \cup \{(\theta_{t+1}, y_{t+1})\}$$

2.4.7. Bayesian Optimization: direct problem

The global exploration stage employs the BO algorithm via *skopt.gp_minimize*, using a custom Gaussian Process Regressor (gpr), defined previously as the surrogate modelling engine for system dynamics evaluations.

The surrogate model is instantiated with internal target normalization. At each optimization step, kernel hyperparameters are updated using maximum marginal likelihood (MML) estimation via the Limited-memory Broyden–Fletcher–Goldfarb–Shanno algorithm (LBFGS). To prevent convergence to sub-optimal local likelihood traps, 10 independent optimization restarts are executed per iteration.

The optimization runs for 150 total evaluations, beginning with 40 random space-filling configurations (Latin Hypercube Sampling) to initialize the GP covariance matrix. The candidate points are then selected by maximizing the Expected Improvement EI acquisition function. An exploration parameter of ξ = 0.05 is assigned to encourage global parameter space exploration around high-uncertainty regions before collapsing search focus around current target peaks. The acquisition

surface is searched continuously using gradient-based optimization (LBFGS), ensuring precise candidate localization across the 9D scaled domain.

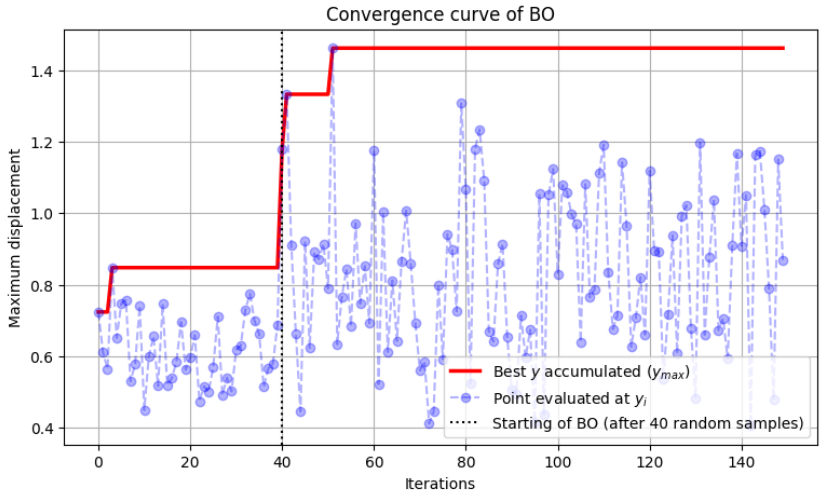


Figure 20

The performance progression for discovering maximum dynamic reach across 150 total evaluation steps is illustrated in the **Convergence Curve**.

During the initial 40 iterations of sampling phase, the maximum reach achieved remains constrained to a nominal plateau of approximately $y_{\max} = 0.85$ m. Upon activating surrogate-guided optimization with Expected Improvement, the algorithm targets high-energy dynamic parameter combinations. Within two iterations after initialization (iteration 42), the accumulated best displacement jumps to 1.34 m. The global maximum configuration is identified at iteration 51, achieving an end-effector peak displacement of 1.46 m. This represents a substantial dynamic reach expansion enabled by resonant coupling with the passive base mass. The continuous spread in evaluated values (0.40 m $<=$ y_i $<=$ 1.30 m) following iteration 51 reflects the acquisition function's balance between local exploitation and global space exploration, confirming that the search space was thoroughly mapped without trapping the surrogate in local optima.

2.4.8. Bayesian Optimization: inverse problem

To determine candidate control and structural configurations capable of achieving prescribed reach demands, an Inverse Optimization Scheme (i.e. using the surrogate model as the function to minimize trough some minimization algorithm)

was executed across a continuous target dynamic displacement range: [0.75 m, 1.70 m] at 0.002 resolution.

Three structural base parameters were locked to fixed physical values (K_b = 4889.12 N/m, M_b = 10.0 kg, h_b = 0.1), leaving a reduced 6-dimensional decision space. For each discrete target, the optimal free parameter vector was found by minimizing the squared error of the Gaussian Process mean prediction and the target in input.

The optimization was solved via L-BFGS-B. Warm-starting was applied between consecutive target steps to ensure parameter path continuity.

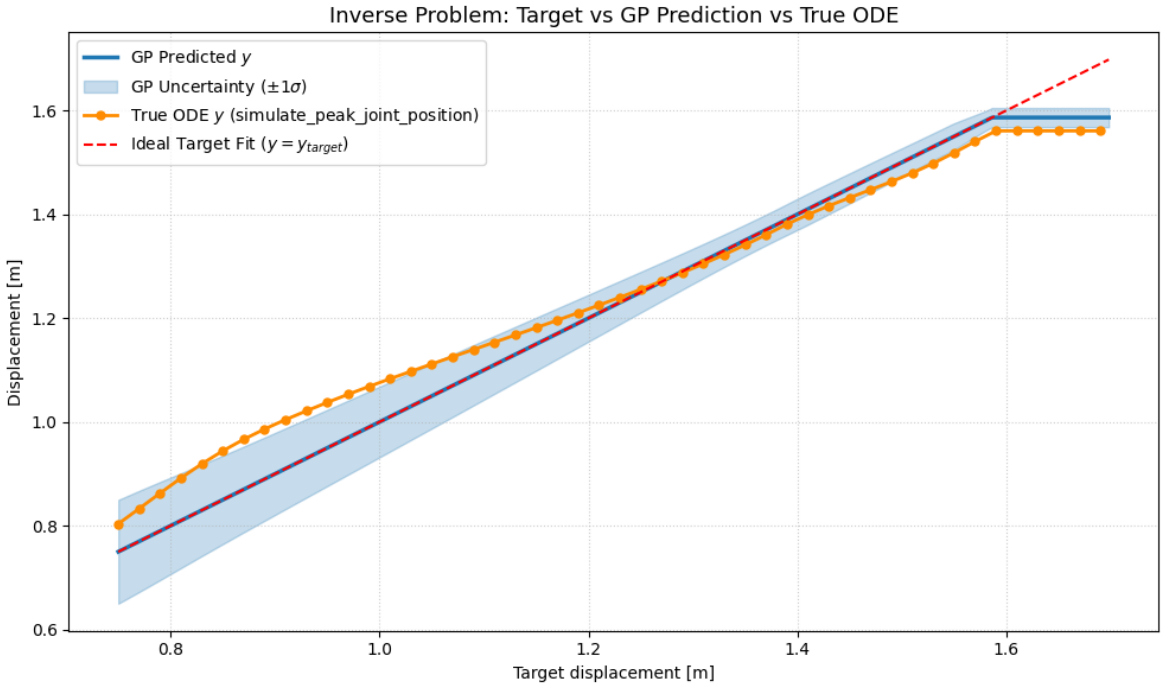


Figure 21

To verify surrogate accuracy, optimized 9D physical parameter sets were sub-sampled (every 10th target step) and executed in the true non-linear 2-DOF dynamic ODE solver. The comparison between target values, surrogate predictions, and true numerical ODE responses is shown in the Inverse Problem Results Plot.

In the range [1.2,1.4] m, the GP surrogate mean matches the target line with near-zero residual error. The ground-truth ODE solver closely tracks this response, remaining entirely within the surrogate's 1-sigma uncertainty envelope. At targets exceeding 1.58 m, both the GP prediction and true ODE response plateau at 1.56 - 1.58 m. This plateau highlights a key physical constraint: under the fixed base parameters (K_b, M_b, h_b), the system reaches its ultimate dynamic resonance limit, preventing further extension regardless of how control parameters are tuned.

To assess the predictive accuracy of the Gaussian Process surrogate model during inverse optimization, a point-wise relative error analysis was performed against the full 2-DOF non-linear numerical ODE solver.

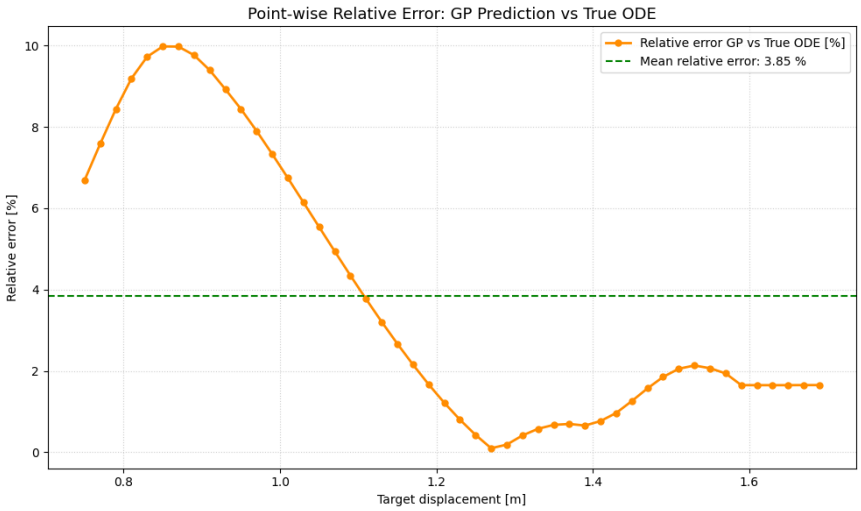


Figure 22

Across the entire evaluated spectrum ([0.75, 1.70] m), the surrogate model achieves an average relative error, confirming that the GP surrogate can substitute for computationally heavy ODE numerical integrators during multi-objective or inverse optimization. Relative error reaches a local maximum of 10.0% around 0.86 m. This behavior stems from lower training sample density in sub-optimal response regions during BO exploration, coupled with the numerical scaling effect of smaller response magnitudes in the percentage error denominator. In the range [1.15, 1.58] m, the relative error drops below 2.0%, reaching a minimum of **0.12%** near 1.27 m. This exceptional precision is a direct benefit of Bayesian Optimization, which densely sampled high-displacement resonant configurations during its acquisition phase. Beyond the system's dynamic capacity limit (1.58 m), the relative error flattens to a constant 1.65%, confirming consistent convergence behavior near the physical boundary of the manipulator.

2.5. Deep Learning

Here an alternative machine-learning approach based on **Artificial Neural Networks (ANNs)** was adapted. Rather than using a Gaussian Process within the Bayesian Optimization framework, a feed-forward Neural Network is trained on a large dataset of simulations generated from the physical model.

The network learns the following nonlinear mapping between the system parameters and the resulting maximum end-effector displacement:

$$f_\Theta: R^d \to R \Rightarrow y_{max} = f_\Theta(\theta)$$

where Θ are the network parameters.

2.5.1. Neural Network architecture

For the surrogate model, a fully connected Multi-Layer Perceptron (MLP) network was constructed.

The architecture employs two hidden layers with 64 and 32 neurons, respectively, activated with Rectified Linear Unit (ReLU) activation functions to capture complex multi-body resonance characteristics.

Mathematically the network is given by:

$$h_1 = ReLU(W_1\theta + b_1)$$
$$h_2 = ReLU(W_2 h_1 + b_2)$$
$$\hat{y}_{max} = W_3 h_2 + b_3$$

The network parameters are given by:

$$\Theta = \{W_1, b_1, W_2, b_2, W_3, b_3\}$$

where W_i and b_i are the weights and biases, respectively.

2.5.2. Direct NN training

To evaluate the convergence behavior of the MLP surrogate model, training and validation MSE losses were tracked across 100 epochs using a logarithmic scale Figure 23.

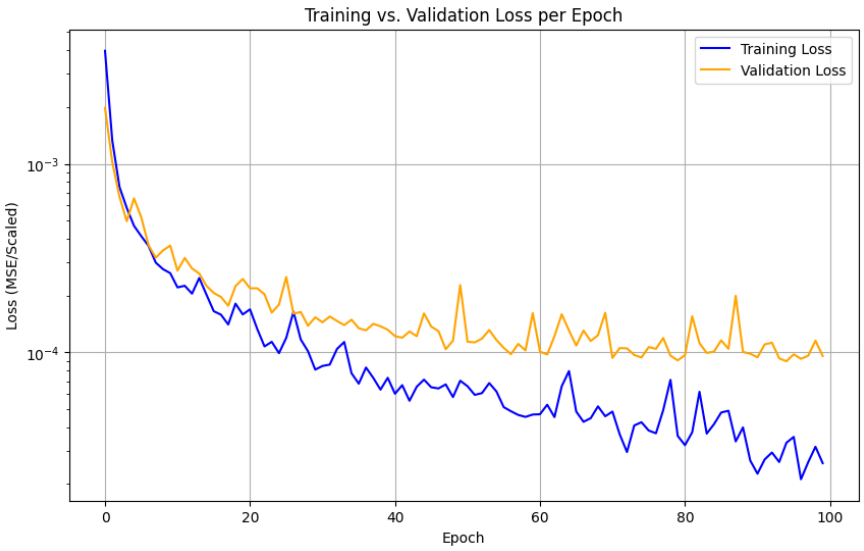


Figure 23

Both loss trajectories exhibit rapid initial convergence within the first 20 epochs, declining by over an order of magnitude. Beyond epoch 40, the validation loss plateaus neatly near 10^{-4}, remaining tightly coupled to the training loss curve. The validation loss does not diverge or curve upwards in the later epochs, confirming the absence of overfitting.

The final validation and test losses remain at quite the same order of magnitude, demonstrating that the surrogate model successfully captures the nonlinear dynamics of the two-mass-spring-damper system and generalizes well across unseen parameter configurations.

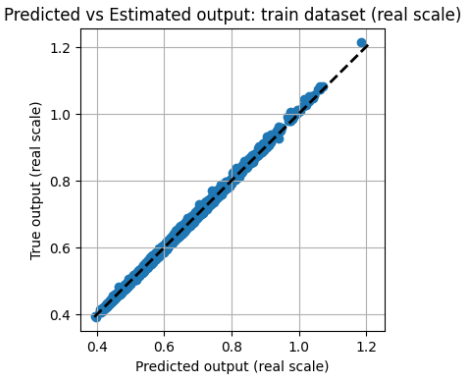


Figure 24

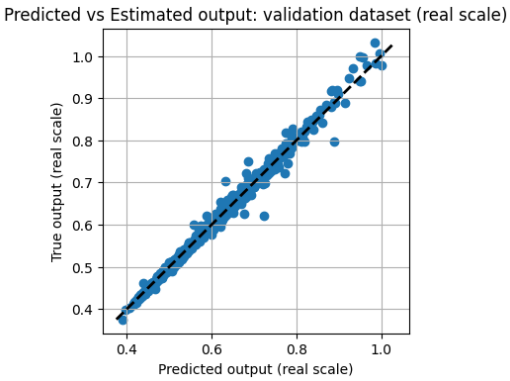


Figure 25

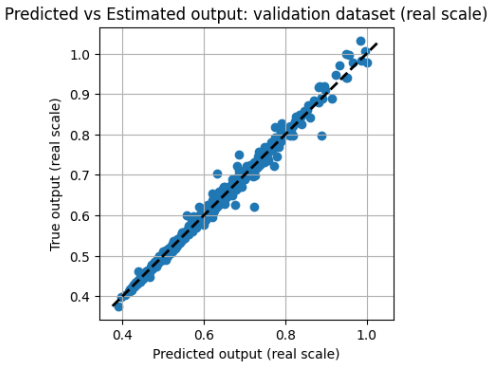


Figure 26

The model's predictive accuracy was evaluated by using statistical metrics such as the RMSE and the coefficient of determination R^2 across training, validation, and test sets.

As shown in the parity plots (Figure 24, 25, 26), predicted joint displacements align tightly with true numerical ODE solutions along the reference line across the entire operating workspace. The strong linear correlation across all data splits confirms that the surrogate model accurately captures nonlinear base-resonance interactions while maintaining high generalization performance on unseen test scenarios.

2.5.3. Sensitivity analysis of the neural network

To evaluate the fidelity and physical consistency of the forward neural surrogate model under structural parameter variations, a sensitivity analysis was conducted on robot damping. For the analysis, a baseline system configuration was selected at random from the test dataset. In the end, the test sample index 1309 was extracted as the reference parameter vector. While keeping all other system parameters fixed, the damping was swept across 10 linearly spaced values from 0.1 (lightly damped) to 1.1 (overdamped). For each step, the surrogate's network predictions were benchmarked against numerical ODE simulations.

In the Figure 27 the resulting response curves and residual profiles from the sensitivity analysis are shown. It can be observed at the top subplot that the surrogate network accurately reproduces the exponential decay curve of the physical ODE solver. As the damping h_r increases from 0.1 to 0.5, peak displacement drops steeply from 1.05 m to 0.60 m before settling into a heavily damped plateau near 0.58 m.

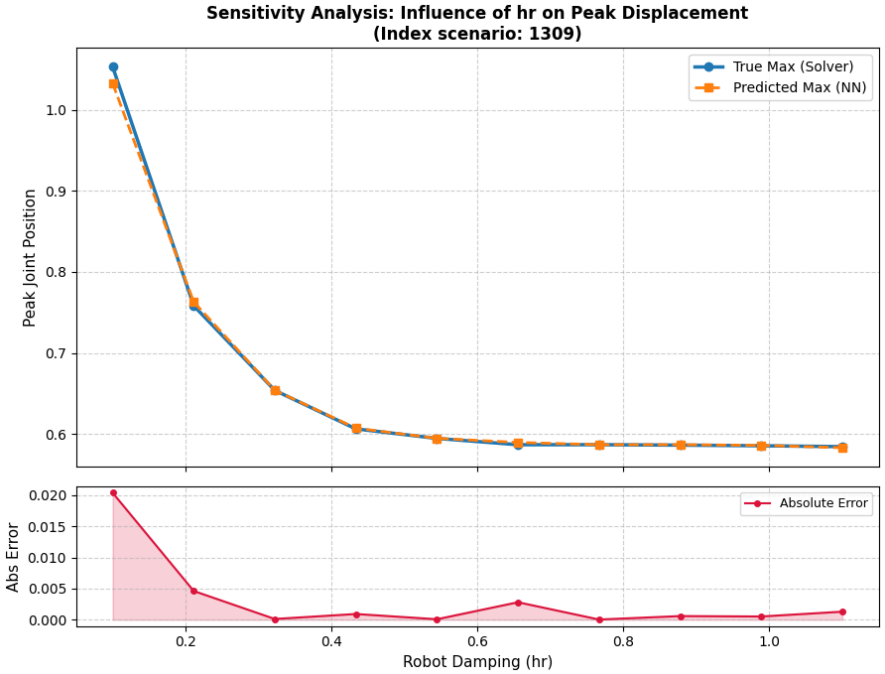


Figure 27

In the lower subplot it can be seen that the absolute residual error is tightly bounded. The maximum absolute error peaks at 0.02 m under low-damping conditions ($h_r = 0.1$), where resonant amplitude gradients are steepest, and quickly decays below 0.005 m for $h_r \geq 0.3$.

2.5.4. Results and analysis

To assess point-by-point surrogate accuracy, point-wise relative errors were computed across the held-out test dataset using physical displacement predictions compared directly against ODE integration results.

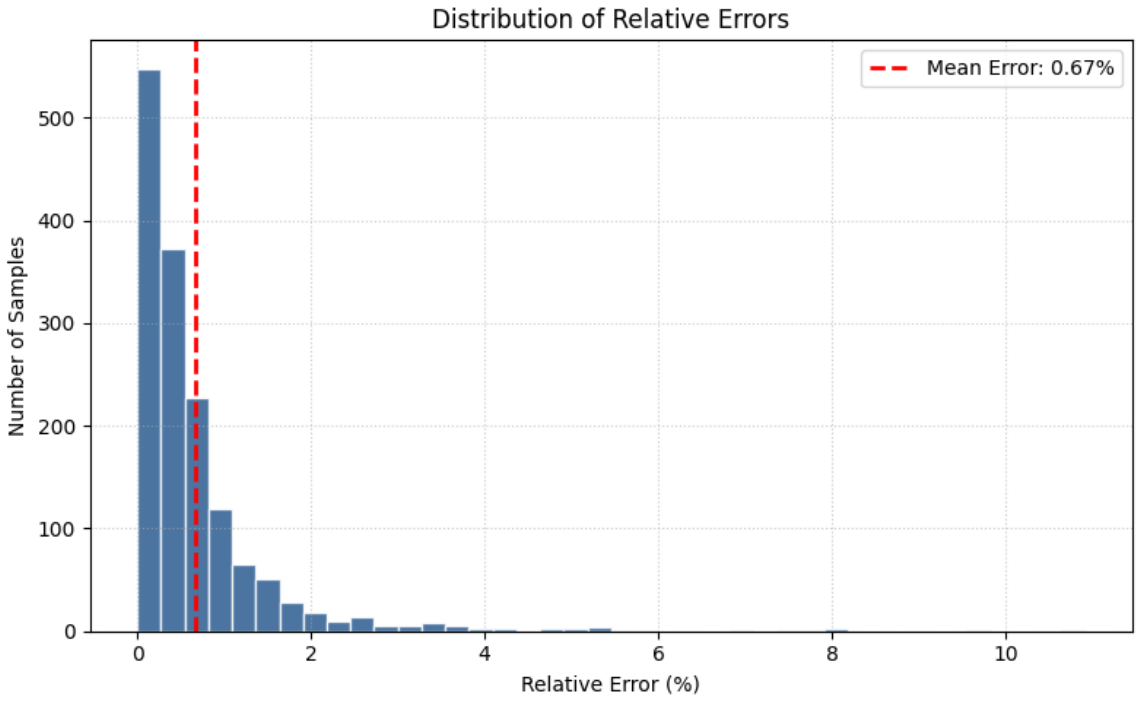


Figure 28

As illustrated in Figure 28, the surrogate model achieves a mean relative error of 0.67%, proving that the forward network captures complex multi-body resonance interactions with high precision. The vast majority of evaluated test conditions yield relative errors well below 2%. Isolated spikes approaching approximately 13% represent edge cases (outliers) associated with low target displacements, where minimal absolute numerical discrepancies inflate percentage error metrics. Overall, the error profile confirms that the neural surrogate provides a fast, highly accurate alternative to numerical ODE integration for downstream control synthesis.

The spatial reliability of the surrogate model was then evaluated by mapping the distribution of the high-residual test samples across the normalized input parameter domain [0, 1] (Figure 29). Samples exceeding one standard deviation (higher than 1σ) and three standard deviations (higher than 3σ) above the mean error were categorized to evaluate boundary effects.

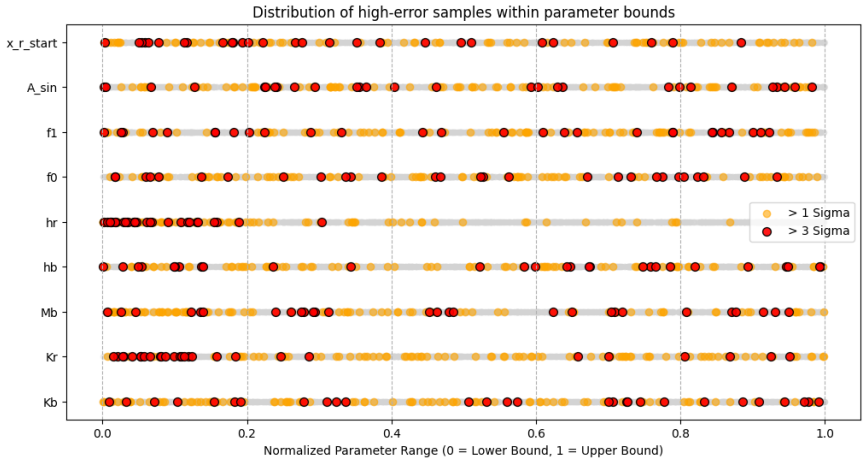


Figure 29

The visualization reveals clear spatial clustering of severe error outliers (errors higher than 3σ) near domain boundaries, most prominently in parameters governing mechanical impedance and geometry (h_r, k_r, x_r). This behavior is characteristic of neural surrogate models, as boundary conditions experience lower local sample density during sampling and sharper gradients in ODE dynamics. Conversely, the central operational domain (0.3 - 0.7) exhibits high fidelity with minimal error spikes, confirming that the model operates with robust accuracy across standard parameter regimes.

2.5.5. Maximum dynamic displacement

A comparative study was performed to assess global optimization performance for maximizing peak joint displacement across the bounded 9-dimensional parameter space.

Initial single-start local optimization using L-BFGS-B initialized at the domain midpoint (x_0) converged prematurely to a local maximum of 0.5674 m. This stagnation occurs due to the non-convex topography of the dynamic response surface, where local gradient algorithms become trapped in sub-optimal basins when lacking global spatial awareness.

To address non-convexity, a Monte Carlo random search evaluating N = 500000 uniform parameter vectors was executed using vectorized tensor batching. The global search identified peak displacements reaching 1.1765 m, which results more than double the local optimizer's output. This demonstrates that global sampling methods or multi-start initialization schemes are essential when navigating complex mechatronic surrogate landscapes.

Parameter	Value
K_b [N/m]	1938.5537
K_r [N/m]	2303.4365
M_b [kg]	94.3593
h_b [-]	0.1166
h_r [-]	0.1010
f_0 [Hz]	0.0048
f_1 [Hz]	5.4579
$A_{\sin}$ [m]	0.1989
$x_{r,\text{start}}$ [m]	0.4928
Maximum displacement of the robot [m]	1.179597

Figure 30

2.5.6. Inverse optimization

To determine the optimal robot parameters set required to achieve a target peak displacement under fixed base conditions k_b, m_b, h_b, an autograd-driven inverse optimization routine was implemented.

Rather than relying on computationally intensive numerical solvers or training an ill-posed inverse neural network, the gradient of the loss function with respect to the continuous input vector was computed via PyTorch automatic differentiation. The network parameters were frozen while the Adam optimizer adjusted the normalized input vector $[0, 1]^6$ over 300 steps.

In a validation case targeting y_{target} = 1.1200 m under fixed base conditions (k_b = 2608.32 N/m, m_b = 98.14 kg, h_b = 0.2690), the optimizer rapidly converged to a surrogate-predicted matching output of 1.1200 m. Forward verification of the synthesized configuration using the true numerical ODE solver produced a peak displacement of 1.1838 m (5.69% relative error), confirming that the autograd optimization routine produces physically valid robot configurations.

The operational range and physical feasibility of the autograd-driven inverse search was then evaluated, for this a continuous target sweep was executed in the range [0.75, 1.50] m of y_{target} under fixed base structural conditions. Each target parameter vector synthesized by the neural surrogate was validated by running direct ODE numerical simulations (Figure 31).

It can be observed that there are two distinct zones in the plot. On the range [0.75, 1.50] m of y_{target}, the autograd and ODE simulations trend exhibits ideal diagonal tracking ($y_{pred} \approx y_{target}$), confirming that the autograd inputs reliably realize desired end-effector displacement in real dynamics. On y_{target} > 1.22 m the surrogate model plateaus near 1.22 m, reaching a physical saturation limit for the specific base configuration, while the true ODE simulations reach a physical upper bound on 1.28 m.

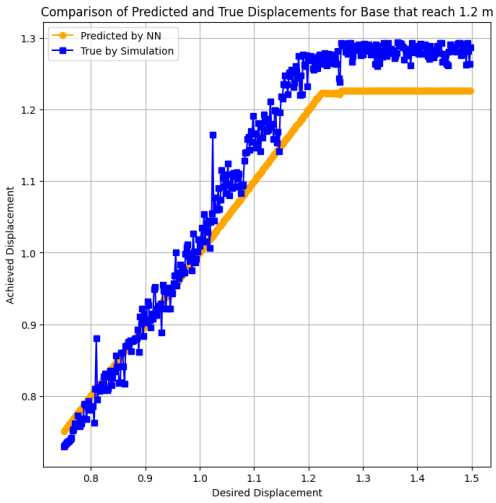


Figure 31

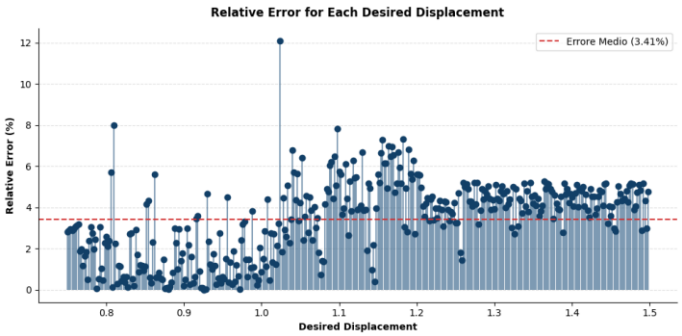


Figure 32

The physical accuracy of the inverse optimization framework was evaluated by calculating the relative error between the predicted targets and the ground-truth ODE simulations. The pipeline achieved a low overall mean relative error of 3.41%, demonstrating that autograd-based input optimization successfully synthesizes physically valid parameter configurations. On the range [0.75, 1.0] m, the relative error remains consistently low at 3%, confirming high surrogate accuracy. Localized error variance increase can be seen on the range [1.0, 1.22] m, reaching an isolated peak of 12.1% near 1.02 m. Lastly on target values > 1.25 m, the target displacements exceed physical capability, stabilizing the relative error into a flat 4-5% band and preventing unbounded error divergence.

Bibliography

[1] L. Roveda, M. Maccarini, F. Pura, F. Reis, and B. Spahiu, "Optimizing Parcels Sorting Through Reinforcement Learning for Intralogistics," in *ECAI 2025*, I. Lynce, Ed., Frontiers in Artificial Intelligence and Applications, vol. 413. Amsterdam: IOS Press, 2025, pp. 5439–5446. doi: 10.3233/FAIA251484.

[2] L. Roveda, N. Pedrocchi, F. Vicentini, and L. Molinari Tosatti, "An interaction controller formulation to systematically avoid force overshoots through impedance shaping method with compliant robot base," *Mechatronics*, vol. 39, pp. 42-53, Oct. 2016.